Sum-of-Subset Problem

- In this problem, we are given a vector of N values, called weights.
- The weights are usually given in ascending order of magnitude and are unique.
- **For example, W= (2, 4, 6, 8, 10) is a weight vector.**
- **We are also given a value M, for example 20.**
- The problem is to find all combinations of the weights that exactly add to M.
- **In this example, the weights that add to 20 are:**

  (2, 4, 6, 8); (2, 8, 10); and (4, 6, 10).

- Solutions to this problem are often expressed by an N-bit binary solution vector, X, where a 1 in position i indicate that W_i is part of the solution and a 0 indicates it is not.
- **In this manner the three solutions above could be expressed as: (1,1,1,1,0); (1,0,0,1,1); (0,1,1,0,1)**

Sum-of-Subsets problem

- We are given ‘n’ positive numbers called weights and we have to find all combinations of these numbers whose sum is M.
- This is called sum of subsets problem.
- If we consider backtracking procedure using fixed tuple strategy , the elements X(i) of the solution vector is either 1 or 0 depending on if the weight W(i) is included or not.
- If the state space tree of the solution, for a node at level I, the left child corresponds to X(i) =1 and right to X(i) =0.

Sum of Subsets Algorithm

void SumOfSub (S, k, r)

{

//Find all subsets of W [1: n] that sums to m

$$\text{//}S=\sum_{j=1}^{k-1} W[j] * x[j] \text{ and } r=\sum_{j=k}^{n} W[j]$$

W[j]' s are in increasing order

// Generate left child.

x[k] = 1;

if (s+w[k] == m)

{

for (int j=1; j<=k; j++)

```
Print (x[j])

}

Else

 if (s+w[k] + w[k+1] <= m)

SumOfSub(s+w[k], k+1, r-w[k]);

// Generate right child and evaluate

 if ((s+r-w[k] >= m) && (s+w[k+1] <= m))

 {

x[k] = 0;

SumOfSub(s, k+1, r-w[k]);

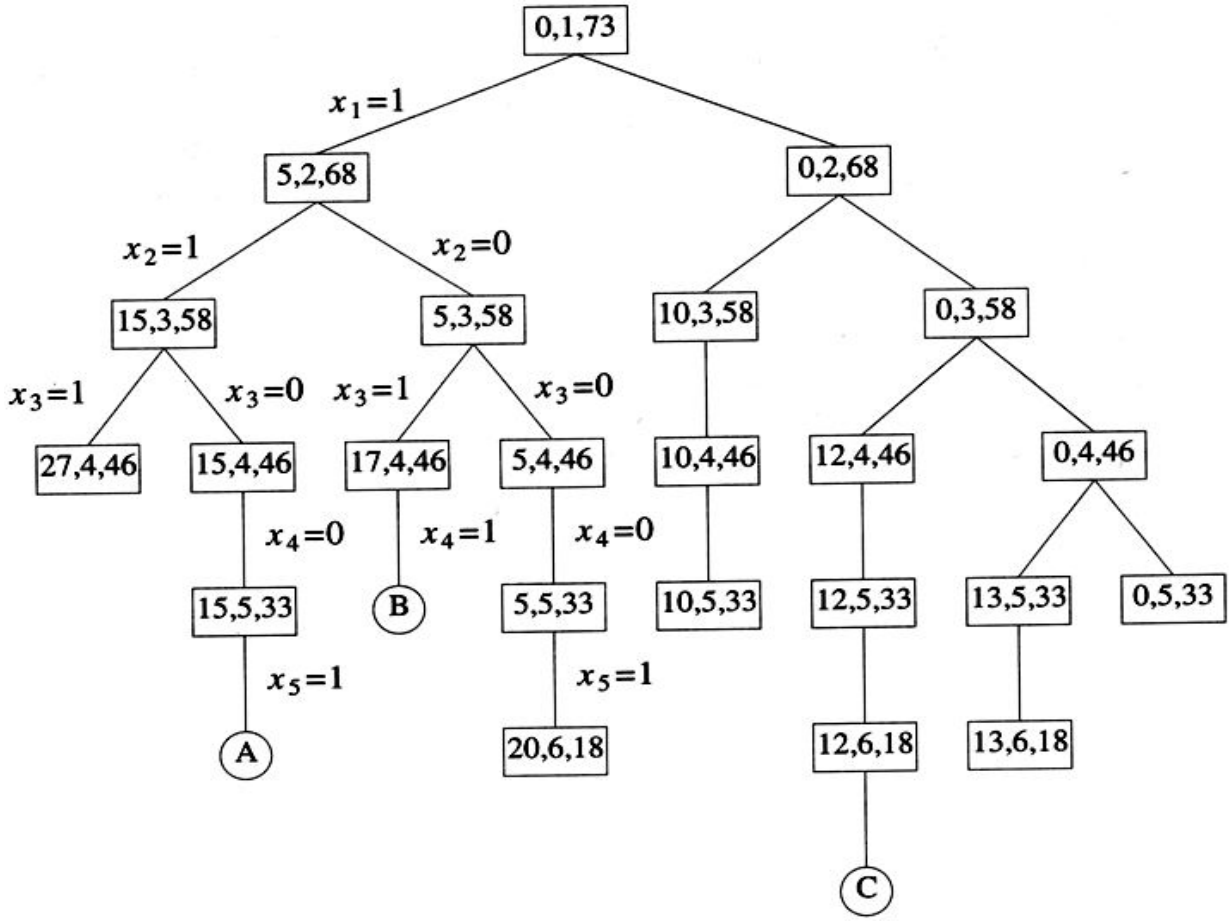
}

}
```

Sum of Subsets State Space Tree

- Example n=6, w[1:6] = {5, 10, 12, 13, 15, 18}, m=30

Sumofsubset(S,k,r)
Initial call
S=0
K=1
r=73



Solutions

A-{1, 1, 0, 0, 1, 0}

B-{1, 0, 1, 1, 0, 0}

C-{0, 0, 1, 0, 0, 1}